Cart Before the Harm

Project Update 2: Complete Literature Review and Dataset Design

Sohan Jayram Reddy Kadiri
Northeastern University
Seattle, WA, USA
kadiri.s@northeastern.edu

Sanidhya Khandelwal
Northeastern University
Seattle, WA, USA

Rohan Raj
Northeastern University
Seattle, WA, USA

Abstract

Commercial jailbreaks ask an LLM shopping assistant to break a business rule, such as granting an unauthorized discount, approving an ineligible refund, or skipping an age check. They share products, policies and vocabulary with legitimate shopping requests, so a benchmark built from unmatched data can be solved from surface wording alone. This second project update makes EcomSafe benchmark-first. We refine the problem and research questions around constructing and validating a policy-grounded benchmark before any detector is trained. We expand the literature review to ten peer-reviewed conference papers, each verified on an official proceedings page. We also describe the single-turn data-generation pipeline we have now implemented. It plans labels before generating text and grounds every prompt in domain policy text. For each commercial attack it generates a matched legitimate request about the same product and policy topic, and it adds legitimate boundary cases. Records then pass rule and near-duplicate filters and a blind judge from a different model family; rejected negatives are repaired, broken pairs are removed, and the data is split into leakage-safe groups and audited for lexical shortcuts. The current pilot, `pilot_150`, contains 126 attacks, 126 matched negatives and 37 boundary cases. A grouped Naive Bayes audit still separates attacks from matched negatives with 0.905 accuracy, which shows that substantial lexical shortcuts remain. This is a dataset-quality warning, not detector performance. No human validation or detector experiments have been run yet. The next cycle focuses on human validation, shortcut reduction and simple detection baselines.

Keywords

LLM safety, jailbreak detection, benchmark construction, e-commerce, matched negatives, dataset artifacts

Project Repository: https://github.com/sohanreddyk/ecomsafe
The repository now contains:

- active development commits;
- the data-generation scripts (`data_generation/`);
- versioned prompt templates, with rendered examples;
- filtering, judging and validation code;
- pilot run summaries with a small record sample;
- an offline test suite.

1 Refined Problem Description and Research Questions

Setting. LLM-based shopping assistants work under business constraints: pricing logic, return and warranty policies, competitor-disclosure rules, and regulatory requirements such as age, identity or prescription checks [6]. A *commercial jailbreak* is a request that tries to make the assistant break one of these rules. Typical goals are an unauthorized discount, a refund the customer is not eligible for, a judgment about a competitor, or bypassing a legal safeguard. The response can look harmless as text while still violating a business rule. General jailbreak benchmarks, which define harm mainly as objectionable content, therefore do not cover this setting [2, 8].

Why the benchmark comes first. Commercial attacks and legitimate requests talk about the same products, prices, return windows and verification steps. An honest customer asking how a price match works and an attacker trying to invent one use much of the same vocabulary. If attack data is collected without matched legitimate counterparts, a classifier can separate the classes using topic words or stereotyped malicious phrasing ("ignore the policy", "make an exception"). Such a classifier would look accurate without recognizing intent. Prior work shows that surface-level dataset artifacts can inflate benchmark scores [4]. Our own pilot audit (Section 3.6) shows that this risk is real for EcomSafe. A detector evaluated on such data would measure lexical cues rather than intent.

Refined problem. EcomSafe's immediate problem is to *construct and validate a policy-grounded benchmark*. The benchmark pairs commercial jailbreaks with legitimate requests about the same product and policy, and adds legitimate boundary cases. Its labels must be reliable, and it must leave as few lexical shortcuts as possible. Once validated, it will support evaluation of lexical, contextual and representation-level detectors, followed by multi-turn trajectory detection. The project is staged accordingly:

- **Phase 1 (current):** benchmark construction and validation.
- **Phase 2 (planned):** single-turn detector evaluation, including hidden-state probes.
- **Phase 3 (planned):** multi-turn generation and trajectory-aware early detection.

The original proposal's hidden-state probe and trajectory layer [6] remain part of the plan for Phases 2 and 3. Neither is implemented, and no detector experiments have been run.

Research questions.

RQ1, benchmark validity. Can policy-grounded synthetic generation, matched negatives, boundary cases, independent model judging and human validation produce reliable commercial-safety intent labels?

RQ2, shortcut resistance. To what extent can attacks and their matched legitimate requests be separated using surface lexical cues, and can improved generation reduce this separability?

Table 1: The ten peer-reviewed conference papers in the core review.

#	Paper	Venue
1	Nakash et al. [9]	EMNLP 2025
2	Mazeika et al. [8] (HarmBench)	ICML 2024
3	Chao et al. [2] (JailbreakBench)	NeurIPS 2024 D&B
4	Russinovich et al. [12] (Crescendo)	USENIX Security 2025
5	Ren et al. [10] (ActorBreaker)	ACL 2025
6	Arditi et al. [1]	NeurIPS 2024
7	Yao et al. [15] (τ-bench)	ICLR 2025
8	Jiang et al. [5] (WildTeaming)	NeurIPS 2024
9	Röttger et al. [11] (XSTest)	NAACL 2024
10	Gururangan et al. [4]	NAACL 2018

RQ3, detection. Given a validated single-turn benchmark, how well do lexical, contextual and representation-level detectors generalize across held-out product domains, attack types and attack techniques?

RQ4, multi-turn (Phase 3, future). Once single-turn validity is established, does trajectory-level information improve early detection of multi-turn commercial jailbreaks?

RQ1 and RQ2 are the focus of the current and next cycle. RQ3 begins once a validated single-turn version is frozen. RQ4 depends on multi-turn generation, which has not been run.

2 Background and Related Work

2.1 Peer-Reviewed Literature

Table 1 lists the ten peer-reviewed conference papers that form our core review. Each venue was verified on an official proceedings page: ACL Anthology, PMLR, NeurIPS, ICLR or USENIX. arXiv-only work is not counted.

1. Effective Red-Teaming of Policy-Adherent Agents [9]. **Scope.** The paper studies agents that must follow strict policies, such as refund or cancellation rules, against users who try to manipulate them into breaking those policies for personal gain. **Methodology.** The authors propose CRAFT, a multi-agent red-teaming system that uses persuasion strategies tailored to the agent's policy. They also introduce tau-break, a benchmark built on τ-bench that measures how well agents resist manipulation, and use it to compare CRAFT with conventional jailbreak methods and to evaluate simple defenses. **Critical Takeaway.** Policy-aware persuasion breaks policy-adherent agents more effectively than generic jailbreaks and the simple defenses tested are insufficient, so commercial-policy violations need their own evaluation.

2. HarmBench [8]. **Scope.** HarmBench is a standardized framework for evaluating automated red-teaming attacks and the robustness of LLM refusals. **Methodology.** The authors specify the properties a red-teaming evaluation needs and build a behavior set and scoring pipeline that meet them. They use it to compare 18 red-teaming methods against 33 target LLMs and defenses, and propose an efficient adversarial-training defense. **Critical Takeaway.** Attack success rates cannot be compared across papers without a shared threat model, behavior set and judge.

3. JailbreakBench [2]. **Scope.** JailbreakBench addresses inconsistent and irreproducible jailbreak evaluation with an open benchmark. **Methodology.** The benchmark provides an evolving repository of jailbreak artifacts, a dataset of 100 misuse behaviors, a standardized evaluation framework with a defined threat model and scoring, and a public leaderboard. **Critical Takeaway.** Jailbreak results are credible only when prompts, code and scoring are released for others to reproduce.

4. Crescendo [12]. **Scope.** The paper introduces Crescendo, a multi-turn jailbreak that opens with benign, abstract questions about a target task and escalates gradually until the model produces content it would refuse if asked directly. **Methodology.** The authors run Crescendo manually against ChatGPT, Gemini Pro and LLaMA variants. They then automate it as Crescendomation, which outperforms prior jailbreaks on an AdvBench subset by 29–61% on GPT-4 and 49–71% on Gemini-Pro. **Critical Takeaway.** Each turn looks benign on its own, so per-message filters struggle to detect gradual escalation even when the attack pattern is known.

5. LLMs Know Their Vulnerabilities (ActorBreaker) [10]. **Scope.** The paper studies how shifting a harmful request toward semantically related but innocuous-looking topics can expose safety gaps in aligned LLMs across multiple turns. **Methodology.** The attack, ActorBreaker, draws on actor-network theory: it identifies "actors" (people, objects, concepts) linked to a harmful target and chains queries about them toward that target. The authors also build a multi-turn safety dataset for fine-tuning defenses. **Critical Takeaway.** An attack can reach a harmful target through related concepts instead of explicit phrasing, so defenses keyed to that phrasing miss it.

6. Refusal Is Mediated by a Single Direction [1]. **Scope.** The paper shows that refusal in open-source chat models is mediated by a single direction in the residual stream: removing it disables refusal, and adding it induces refusal on harmless prompts. **Methodology.** The direction is extracted as the difference in mean activations between harmful and harmless instructions, and is tested causally through directional ablation and activation addition. A rank-one weight edit based on it yields a white-box jailbreak. **Critical Takeaway.** Safety behavior sits in a low-dimensional, fragile internal structure, which makes it easy to remove and also readable by probes.

7. τ-bench [15]. **Scope.** τ-bench evaluates language agents that talk with simulated users while following domain-specific policy guidelines and using API tools, in retail and airline customer-service domains. **Methodology.** An LM simulates the user. Success is judged by comparing the final database state with an annotated goal state, and a pass$\hat{}k$ metric measures how reliably an agent succeeds across repeated trials. **Critical Takeaway.** Even strong function-calling agents follow domain rules inconsistently across repeated trials, so policy adherence must be measured directly instead of assumed.

8. WildTeaming [5]. **Scope.** WildTeaming is an automatic red-teaming framework that mines in-the-wild user–chatbot interactions to discover jailbreak tactics and composes them into new adversarial attacks. The authors use it to build WildJailbreak, a large open safety-training dataset. **Methodology.** WildJailbreak

contains two contrastive kinds of queries: harmful queries, both plain and adversarial, and benign queries that resemble harmful ones in form but carry no harmful intent. The authors train models on it and evaluate both safety and over-refusal. **Critical Takeaway.** A safety dataset needs benign counterparts that look adversarial in form, or a model can learn the surface form of an attack rather than its intent and over-refuse benign requests.

9. XSTest [11]. **Scope.** XSTest is a test suite for exaggerated safety, where models refuse clearly safe prompts because they superficially resemble unsafe ones. **Methodology.** The suite pairs safe prompts from several prompt types (for example homonyms, figurative language and safe contexts) with contrasting unsafe prompts, and uses them to characterize refusal behavior across models. **Critical Takeaway.** Models refuse safe requests that share vocabulary with unsafe ones, so safety evaluation needs explicit near-boundary benign cases.

10. Annotation Artifacts in NLI Data [4]. **Scope.** The paper shows that crowd-sourced natural language inference datasets contain annotation artifacts: lexical and stylistic cues in the hypotheses that reveal the label without the premise. **Methodology.** The authors train a text classifier on hypotheses alone, find that it predicts the label far above chance, analyze the cue words involved, and re-evaluate models on the "hard" subset that the hypothesis-only classifier gets wrong. **Critical Takeaway.** When a classifier that sees only surface features succeeds, benchmark scores overstate understanding, so datasets should be audited and reported with such baselines.

Recent supporting preprints (not counted). The following arXiv preprints inform our design. None has a verified peer-reviewed venue, so none counts toward the ten.

- **SIA** [16]: synthesizes safety-aware training data, including adversarial examples, for e-commerce search LLMs. It is the closest industrial evidence that e-commerce assistants face jailbreak and compliance risk, and it inspired our boundary cases.
- **Organizational Control Layer** [14]: checks an agent's proposed economic actions against role, policy and economic constraints before they execute. It is a complementary, execution-time defense for price and refund governance.
- **Multi-turn safety** [3, 7, 13, 17]: Cognitive Firewall, Multi-TurnPSB, State-Dependent Safety Failures and Speak Out of Turn. These motivated the Phase 3 extension in Update 1.

2.2 Thematic Synthesis

Policy-adherent agent attacks. τ-bench establishes customer-service agents bound by written policies as an evaluation setting [15]. CRAFT and tau-break show that persuasion grounded in those policies breaks such agents more often than generic jailbreaks [9]. EcomSafe follows this line: every generated attack targets a specific written policy fragment of a fictional store. Unlike these works, EcomSafe studies detection of shopper intent at the request level rather than end-to-end agent compliance.

Red-teaming benchmarks and reproducibility. HarmBench and JailbreakBench argue that results need a fixed threat model, behavior set and judge, together with released artifacts [2, 8]. WildTeaming shows how to scale adversarial data while keeping it organized by tactic [5]. EcomSafe applies these principles to data construction: a fixed taxonomy, versioned prompt templates, a recorded configuration for each run, per-stage rejection logs, and public generation code.

Multi-turn attack dynamics. Crescendo and ActorBreaker show that harmful goals can be reached through gradual escalation or through related topics, with each turn looking benign [10, 12]. These results motivate Phase 3. They are also why Phase 1 is explicitly limited to single-turn requests: a single-turn benchmark cannot by itself measure escalation.

False positives and benign counterparts. XSTest shows that models over-refuse safe prompts that resemble unsafe ones [11]. WildTeaming pairs harmful queries with benign queries of similar form [5]. EcomSafe borrows both ideas: matched negatives mirror the form of each attack, and boundary cases probe over-blocking. In commerce, false positives have a direct business cost, because blocking an honest refund or price-match request harms a legitimate customer.

Representation-level safety signals. Arditi et al. show that refusal is mediated by a linearly readable direction [1]. This motivates the Phase 2 hypothesis that a linear probe over hidden states can recover commercial intent. Matched negatives are what make that test meaningful, because a probe that separates attacks from matched negatives cannot rely on topic alone. The same work also notes that the signal is fragile under white-box manipulation.

Dataset shortcuts and artifacts. Gururangan et al. show that a surface-only classifier can expose artifacts that inflate benchmark scores [4]. EcomSafe builds this check into its pipeline: every run reports how well a lexical classifier separates attacks from matched negatives. A high score is treated as a defect in the data.

2.3 Refined Research Gap

Each ingredient of EcomSafe has precedent, and we do not claim otherwise:

- general jailbreak and harmful-content benchmarks [2, 8];
- red-teaming of policy-adherent agents [9, 15];
- multi-turn jailbreaks [10, 12];
- over-refusal and benign counterparts [5, 11];
- representation-level safety signals [1];
- artifact audits [4].

Trajectory-aware detection and hidden-state probing are also not new. EcomSafe's contribution is at the intersection of these lines of work. It combines:

- an explicit taxonomy of e-commerce business-rule violations;
- attacks paired with legitimate requests about the same product and policy;
- legitimate boundary cases;
- held-out evaluation by product domain, attack type and technique;
- a lexical shortcut audit as a standard quality check;

Table 2: Planned EcomSafeBench composition from the proposal [6]. This is a design target and has not been built.

Subset	Conv.	Content
Commercial jailbreaks	1,300	Instances of the five attack types
Multi-turn escalation	1,100	Linear, camouflaged, late spike
Benign / control	1,500	Customer-service and shopping dialogues
Total	3,900	

- a planned extension to trajectory-aware detection.

Among the works we reviewed, none provides this combination for commercial safety.

3 Dataset and Pre-processing

This section separates three things: the *planned* full benchmark (Section 3.1), the *implemented* single-turn pipeline (Sections 3.2–3.6), and the *observed* pilot data (Section 3.7).

3.1 Original EcomSafeBench Design

The proposal specifies EcomSafeBench as 3,900 annotated conversations across five product verticals: electronics, pharmaceuticals, fashion, financial products and groceries [6]. Table 2 gives the planned subsets. The multi-turn subset uses three escalation patterns: linear, camouflaged and late spike. The planned evaluation covers:

- ROC AUC per threat class;
- early-detection rate (attacks caught before turn 5);
- false-positive rate on benign dialogues;
- per-turn latency.

The full 3,900-conversation benchmark has not been constructed. What exists is a single-turn generation and validation pipeline and small pilot datasets.

3.2 Implemented Single-Turn Data-Generation Pipeline

The implemented pipeline is `ecomsafe_datagen`, located in the repository's `data_generation/` folder. It synthesizes Threat Class 1: single-turn commercial jailbreaks. Table 3 lists its stages.

Shopper text only. The generator writes only what the shopper types. It never writes an assistant reply or instructions for carrying out a violation. Each current record therefore contains exactly one shopper (user) message and no assistant turn.

Labels before text. For each record, `plans.py` fixes the labels before any text exists:

- intent, severity, attack type and framing technique;
- domain, product and the policy topic being targeted.

The language model only writes text that realizes this plan, so the ground truth comes from the design. The judge then checks whether the text matches it.

Policy grounding. `domains.py` defines a small fictional catalog for each of the five verticals, with no real brands. It also holds policy text by topic (pricing, returns, warranty, competitor, restricted) and weights that control how often each attack type is sampled in each domain. For example, regulatory bypass has weight 0 in fashion, so that pairing is never sampled. Every generation prompt includes the exact policy text that the attack targets.

Models used in the pilots. Both models are served by Groq:

- **Generator:** `openai/gpt-oss-120b`, temperature 0.9, reasoning effort "medium", five variants per call.
- **Judge:** `qwen/qwen3.8-27b`, a different model family, with reasoning disabled.

The code supports other backends, but none were used in the reported pilots. All LLM calls are cached, prompts are versioned (`st-v2.4` for the headline pilot), and each run records the exact configuration it used.

3.3 Threat Taxonomy and Labels

Attack types and intents. Table 4 lists the five commercial attack types and the shopper intent label each maps to. The mapping is many-to-one: price manipulation and policy bypass both map to PV. The four attack intents are:

- **PV**, Policy Violation;
- **CD**, Competitive Disclosure;
- **FF**, Fraud Facilitation;
- **RB**, Regulatory Breach.

Matched negatives and boundary cases carry the label **BENIGN**.

Response labels. Compliant Refusal (CR) is *not* a shopper intent. It belongs to a separate set of response labels (CR, COMPLIED, PARTIAL, HELPED) that describe how a target model responds. These labels will be assigned only after target-model responses are collected in Phase 2.

Techniques. Each attack is wrapped in one of eight framing techniques:

- `direct`;
- `persona_override`;
- `authority_claim`;
- `policy_misquote`;
- `emotional_appeal`;
- `coercion_threat`;
- `hypothetical_framing`;
- `instruction_injection`.

Each technique also has a defined benign analogue, which matched negatives can reuse.

Severity and risk target. Severity runs from 0 (none) through 1 (probing) and 2 (clear but implicit) to 3 (explicit). Single-turn attacks are planned with severity 2 or 3 in a 50/50 mix, and benign records have severity 0. Each user turn stores

$$risk_target = w(intent) \times s(severity),$$

with weights w: FF = 1.0, RB = 0.8, PV = 0.6, CD = 0.4, and severity scale s: 0, 0.33, 0.67, 1.0.

- The weights come from the proposal's per-turn risk equation (Eq. 1) [6].

Table 3: Implemented pipeline stages (`data_generation/ecomsafe_datagen/`).

Stage	Module	Function
Label-first planning	`taxonomy.py`, `plans.py`	Fix intent, severity, attack type, technique, domain, product and policy topic before generation
Domain-policy grounding	`domains.py`	Fictional catalogs, policy text by topic, attack-type weights by domain
Attack generation	`prompts.py`, `generate.py`	Five variants per call; refusals detected and re-sampled
Matched negatives	`prompts.py`, `generate.py`	One legitimate counterpart per attack: same product and policy topic, mirrored form
Boundary cases	`prompts.py`, `generate.py`	Legitimate requests near a policy boundary
Rule filter	`quality.py`	Length (15–700 characters), refusal text, banned meta-words, label tokens
Near-duplicate filter	`quality.py`	MinHash, Jaccard $\geq$ 0.8 on character 5-grams, applied within each class
Blind judge (+ repair)	`quality.py`, `pipeline.py`	Independent relabeling without the intended label; rejected matched negatives rewritten for up to 2 rounds
Pair integrity, trimming	`pipeline.py`	Drop records whose partner did not survive; trim attacks to target, stratified by domain $\times$ attack type
Splits	`splits.py`	Grouped 70/10/20 train/val/test; held-out domain, technique and attack type
Shortcut audit	`quality.py`	Grouped 5-fold Naive Bayes on unigrams and bigrams, attack vs. matched negative
Annotation tooling	`annotate.py`	Blind sample export; Cohen's κ human vs. design and human vs. judge
Preflight	`preflight.py`	Config, key and model-family checks; live connectivity; optional smoke run with invariant checks

Table 4: Implemented commercial attack types (Threat Class 1).

Attack type	Intent	Policy topic: goal
Price manipulation	PV	Pricing: override prices, invent or stack discounts
Policy bypass	PV	Returns or warranty: an exception the shopper does not qualify for
Return fraud	FF	Returns: guidance on deceiving the store, such as false damage claims or avoiding detection
Competitor extraction	CD	Competitor: rival rankings, or internal margins and unannounced price changes
Regulatory bypass	RB	Restricted: skip age, ID, prescription or KYC checks, or obtain dosing beyond the label

- The severity scaling, and its use as a supervised per-turn target, is a new extension in our implementation.

3.4 Matched Negatives and Boundary Cases

We use four record categories:

- **Attack:** a shopper request intended to violate, or to help violate, a commercial policy.
- **Matched negative:** a legitimate request paired on purpose with one attack. It concerns the same product and the same policy topic.
- **Boundary case:** a legitimate request near a policy boundary, such as an honest price-match request or a return just outside the window. It is designed to test over-blocking.
- **Benign/control:** the umbrella category for all legitimate records. Matched negatives and boundary cases are both BENIGN; neither is an attack.

How matched negatives are written. The generator sees the attack text and writes a counterpart that mirrors its sentence form and reuses its harmless wording. In a configured share of pairs (0.8 in the headline pilot), the counterpart also keeps the attack's framing in a legitimate form. Examples are an accurate policy quotation instead of a misquote, or a real role mentioned for context.

Per-type guards forbid the counterpart from keeping any part of the forbidden request, and prompts require it to replace any false claim about the policy. The aim is to make topic, vocabulary and framing poor predictors of the label, so that the classes differ mainly in intent. The pipeline does not yet achieve this aim (Section 3.6).

3.5 Filtering, Judging, Repair, and Pair Integrity

Deterministic filters. The rule filter removes messages that are too short or too long (15–700 characters), contain refusal text, contain banned meta-words (for example "jailbreak" or "dataset"), or leak label tokens.

The near-duplicate filter uses MinHash locality-sensitive hashing with a Jaccard threshold of 0.8 on character 5-grams. It runs *within* each class (attacks, matched negatives, boundary cases). An attack and its matched negative are similar by design, so neither is removed for resembling the other.

Blind judge. An independent judge from a different model family relabels each message. It sees:

- the store domain;
- the relevant policy text;
- the message.

It never sees the intended label. It returns an intent, a severity, a realism flag, a confidence score and a short rationale. A record is accepted when:

- the judge's intent matches the planned intent;
- its confidence is at least 0.5;
- it marks the message as realistic.

The configuration treats PV↔FF as an acceptable soft agreement. In the headline pilot, every accepted record had full agreement.

Matched-negative repair. If the judge rejects a matched negative whose attack was accepted, the negative is rewritten using the judge's rationale as feedback and judged again, for up to two rounds. This raises the number of complete pairs. It also creates a possible *judge-conditioning bias*: repaired records are optimized to pass the same judge that later accepts them, so judge agreement is not independent evidence of their validity. Human validation is required to resolve this (Section 4.1).

Pair integrity and trimming. With orphans disabled, an attack or matched negative is dropped if its partner did not survive. Attacks are then trimmed to the run's target, stratified by domain × attack type. The pipeline records each dropped record and the stage that dropped it, with one exception: trimmed records are not yet logged (Section 3.8).

3.6 Splits and Shortcut Auditing

Leakage-safe splits. Records are split 70/10/20 into train, validation and test. The split unit is a connected component over two links:

- `gen_group`, which links the five variants produced by one generator call;
- `pair_id`, which links an attack to its matched negative.

Near-paraphrases and minimal pairs therefore never straddle partitions. Components are stratified by domain, attack type and record kind. Three held-out splits are configured for generalization (RQ3). In each, the held-out value goes entirely to test:

- domain `groceries`;
- technique `instruction_injection`;
- attack type `competitor_extraction`.

Lexical shortcut audit. Each run trains a multinomial Naive Bayes classifier on unigram and bigram features to separate attacks from matched negatives. Evaluation is 5-fold cross-validation grouped by `gen_group`, and boundary cases are excluded. The classifier sees only surface wording. **High accuracy therefore indicates bad dataset quality**, because it means the classes can be told apart without understanding intent. The audit is a diagnostic of the data, not an EcomSafe detector, and its accuracy is not detection performance.

3.7 Current Pilot Dataset Statistics

Table 5 summarizes the headline pilot, `pilot_150` (prompt version `st-v2.4`, attack target 150), and the preceding iteration, `pilot_v5` (`st-v2.3`, target 50). Both are pipeline pilots, not the final benchmark.

Funnel. In `pilot_150`, 392 messages were generated. None was removed by the rule or near-duplicate filters, which suggests these filters are permissive for the current prompts. After judging and repair, 375 remained. In total, 17 records were rejected for judge disagreement: 4 attacks, 10 matched negatives and 3 boundary cases. Pair integrity removed 14 orphans, and stratified trimming reduced the set to the final 289.

Shortcut finding. In `pilot_150`, the grouped 5-fold multinomial Naive Bayes audit, using unigram and bigram features, separated attacks from matched negatives with 0.905 accuracy ($n = 252$). The audit intentionally uses only surface lexical information. This result therefore indicates that substantial lexical shortcuts remain in the current pilot, and it motivates another iteration of matched-negative and attack phrasing. The most attack-indicative n-grams include *exception*, *bypass*, *override*, *competitor* and *margin*. The most negative-indicative include *price_match*, *catalog* and *what_verification*. Attacks still announce their intent with stereotyped words, while negatives fall back on stock information-seeking phrasing. The score was 0.84 on the smaller `pilot_v5`, so the problem did not shrink as the pilot grew.

Product verticals. Table 6 breaks down `pilot_150` by vertical. Coverage is uneven:

- Fashion has only 9 attacks, and they cover only two attack types. Regulatory bypass is excluded there by design.
- Financial products and groceries contain no return-fraud attacks.
- Boundary cases range from 4 (pharmaceuticals) to 15 (groceries).

Technique distribution. All eight techniques appear among the `pilot_150` attacks:

- 28 `policy_misquote`;
- 27 `hypothetical_framing`;
- 23 `instruction_injection`;
- 20 `coercion_threat`;
- 10 `direct`;
- 7 `authority_claim`;
- 7 `emotional_appeal`;
- 4 `persona_override`.

Turn lengths.

- **Current data:** every record has exactly one shopper turn and zero assistant turns. Messages average 32.0 words (median 31) in `pilot_150`.
- **Multi-turn scaffolding:** plan builders for linear, camouflaged and late-spike conversations, history-conditioned generation and per-turn judging are implemented. The configuration specifies 3–7 user turns, a 0.4/0.3/0.3 pattern mix and a target of 1,100 conversations, but multi-turn generation is disabled. **No multi-turn data has been generated.**
- **Open design issue:** the proposal's early-detection metric counts attacks caught "before turn 5", yet the configured 3–7 turn range allows conversations shorter than five turns. We record this conflict for the next design iteration rather than resolving it here.

Table 5: Observed pilot statistics. "Lexical audit" is grouped Naive Bayes accuracy on attack vs. matched negative; higher means more lexical shortcuts, so it is not detector accuracy.

Run	Generated	After judge	After pairing	Final	Attacks	Matched neg.	Boundary	Train/Val/Test
`pilot_v5`	155	149	144	114	50	50	14	81 / 12 / 21
`pilot_150`	392	375	361	289	126	126	37	195 / 35 / 59
Lexical audit: `pilot_v5` 0.84 ($n = 100$); `pilot_150` 0.905 ($n = 252$).								

Table 6: `pilot_150` by product vertical. Attack-type columns count attacks. Matched negatives equal attacks in each vertical. Abbreviations: PM price manipulation, PB policy bypass, RF return fraud, CE competitor extraction, RBy regulatory bypass, Bd boundary cases.

Vertical	PM	PB	RF	CE	RBy	Att.	Bd	Total
Electronics	5	9	9	9	4	36	5	77
Pharmaceuticals	0	0	3	8	15	26	4	56
Fashion	0	0	4	5	–	9	5	23
Financial prod.	5	10	0	0	8	23	8	54
Groceries	10	8	0	6	8	32	15	79
Total	20	27	16	28	35	126	37	289

Organic data. The pipeline can load real customer-service text from the Bitext and Twitter customer-support corpora as organic benign data, but both input paths are unset. **All pilot data is synthetic.**

3.8 Current Limitations

- **Lexical shortcuts.** The 0.905 audit accuracy means the pilot is not yet usable for detector evaluation without further iteration.
- **Generator refusals.** The generator often declines to write return-fraud and instruction-injection messages:
  - `pilot_150`: 22 of 26 return-fraud attempts and 11 of 16 instruction-injection attempts were refused, and six attack plans failed entirely.
  - `pilot_v5`: the corresponding figures were 4 of 8 and 3 of 7.

  These denominators count API attempts, including retries. They are not counts of unique or final records and should not be read as dataset percentages.
- **Repair bias.** 47 of the 126 final matched negatives in `pilot_150`, and 25 of 50 in `pilot_v5`, were rewritten using feedback from the judge that then accepted them.
- **No human validation yet.** The annotation tooling exists, but no annotation has been run. Agreement between two human annotators and scoring of the realism answers are not yet implemented.
- **Single judge.** Generator and judge come from different model families, but agreement with one model judge is not human agreement.
- **Coverage and scale.** The pilots are small, coverage across verticals and attack types is uneven, and the configured target of 1,000 attacks has not been attempted.
- **Bookkeeping.** Records removed by trimming are not yet written to the rejection log.
- **No multi-turn or organic data** (Section 3.7).

4 Plan for Next Weeks

The plan follows from the problems observed. Labels must be validated and shortcuts reduced before a single-turn version is frozen and scaled. Simple baselines are built in parallel on frozen pilot data, as sanity checks only.

4.1 Dataset Annotation and Validation

(1) **Annotation protocol.** Write labeling guidelines covering the PV/CD/FF/RB/BENIGN definitions, the PV-vs-FF and PV-vs-RB disambiguation rules, and realism. Draw a stratified sample of about 100 `pilot_150` records. The sample should oversample repaired matched negatives and boundary cases.
(2) **Independent annotation.** Two annotators label every item without seeing the design label or the judge label.
(3) **Agreement metrics.** Add human-vs-human agreement (Cohen's κ) to `annotate.py`, and score the realism answers. Keep the existing human-vs-design and human-vs-judge κ and confusion matrix.
(4) **Adjudication.** Resolve disagreements by discussion and record the decisions. Report agreement separately for repaired negatives.
(5) **Shortcut reduction.** Revise attack and matched-negative prompts so that attacks do not rely on stereotyped words and negatives do not fall back on stock questions. Add an attack-vs-boundary audit, and re-measure until the lexical audit falls substantially.
(6) **Refusals and coverage.** Reduce return-fraud and instruction-injection refusals, and record per-record generator provenance. Rebalance vertical and attack-type coverage. Log records removed by trimming.
(7) **Freeze and scale.** Freeze a validated single-turn version, then scale toward the configured target. Multi-turn generation is enabled only after this, once the turn-5 metric conflict is resolved.

4.2 Probe Baseline Implementation

Baselines are built from simple to complex. All of them run on frozen pilot data, and pilot-scale results will be reported only as sanity checks.

(1) A Naive Bayes lexical baseline, reusing the audit as a reference point.

Table 7: Proposed task distribution for the next cycle: planned team workstreams. All workstreams are shared group work.

Workstream	Next-cycle tasks	Deliverable
Dataset validation	Annotation guidelines; independent human annotation of about 100 `pilot_150` records by two annotators; human-vs-human agreement and realism scoring added to `annotate.py`; disagreement adjudication	Validated pilot labels with κ (human–human, human–design, human–judge) and confusion matrices
Dataset refinement	Reduce lexical shortcuts; improve matched-negative wording; address return-fraud and instruction-injection refusals; balance vertical and attack-type coverage; log trimmed records	Frozen, versioned single-turn dataset with a lower lexical audit score
Probe baselines	Naive Bayes and TF-IDF logistic regression; sentence-embedding classifier; Llama-3.2-3B-Instruct hidden-state extraction; linear (logistic) probe	Baseline comparison on the grouped train/validation/test split
Generalization	Held-out domain, attack-type and technique evaluation; false-positive rate on boundary cases	Evaluation report (pilot-scale sanity checks)
Multi-turn preparation	Resolve the turn-length vs. "before turn 5" metric conflict; tune escalation prompts; plan benign multi-turn dialogues	Multi-turn generation plan
Documentation and reproducibility	Versioned configs, prompt templates and run reports in the repository; offline tests; report writing	Updated repository and next project update

(2) TF-IDF logistic regression.
(3) A sentence-embedding classifier.
(4) Hidden-state extraction from Llama-3.2-3B-Instruct, the proposal's smaller target model [6], on each shopper message.
(5) A linear (logistic) probe on the extracted states.

Each baseline will be evaluated on:

- the grouped train/validation/test split;
- the three held-out splits (domain, attack type, technique);
- the false-positive rate on boundary cases.

No complex neural or trajectory detector will be built before these baselines exist.

4.3 Proposed Team Task Distribution

Table 7 gives the proposed task distribution for the next cycle as planned team workstreams. All workstreams are shared group work, and the table is a plan, not a record of past work.

5 Conclusion

Update 2 moves EcomSafe from a proposed benchmark to an implemented single-turn pipeline for constructing it. The pipeline:

- plans labels before generating text;
- grounds every request in written store policy;
- pairs each commercial attack with a matched legitimate request and adds boundary cases;
- applies deterministic filters, a blind judge from a different model family, repair and pair integrity;
- produces leakage-safe grouped and held-out splits;
- audits each run for lexical shortcuts.

We also expanded the related work to ten verified peer-reviewed papers and narrowed the research gap to their intersection.

The pipeline works end to end. However, the 0.905 lexical audit on `pilot_150`, together with generator refusals, possible repair bias and the absence of human validation, shows that the data is not yet ready to be treated as a final benchmark. The next cycle focuses on human validation, shortcut reduction and freezing a single-turn benchmark, while simple lexical, embedding and hidden-state baselines are established. Multi-turn trajectory experiments follow after that.

Acknowledgments

We thank Jayash Koshal and Shanu Sushmita of Northeastern University for their guidance and feedback throughout the EcomSafe project.

References

[1] Andy Arditi, Oscar Obeso, Aaquib Syed, Daniel Paleka, Nina Panickssery, Wes Gurnee, and Neel Nanda. 2024. Refusal in Language Models Is Mediated by a Single Direction. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. doi:10.52202/079017-4322

[2] Patrick Chao, Edoardo Debenedetti, Alexander Robey, Maksym Andriushchenko, Francesco Croce, Vikash Sehwag, Edgar Dobriban, Nicolas Flammarion, George J. Pappas, Florian Tramèr, Hamed Hassani, and Eric Wong. 2024. JailbreakBench: An Open Robustness Benchmark for Jailbreaking Large Language Models. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track*. doi:10.52202/079017-1745

[3] M. Guida, R. Shikhhamzayev, S. Penchala, S. Iannucci, J. Li, S. Rahimi, and N. Amiri Golilarz. 2026. Cognitive Firewall: A Proactive, Zero-Trust, Multi-Gate Framework for LLM Safety. arXiv:2607.01277 arXiv preprint.

[4] Suchin Gururangan, Swabha Swayamdipta, Omer Levy, Roy Schwartz, Samuel R. Bowman, and Noah A. Smith. 2018. Annotation Artifacts in Natural Language Inference Data. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*. Association for Computational Linguistics, 107–112. doi:10.18653/v1/N18-2017

[5] Liwei Jiang, Kavel Rao, Seungju Han, Allyson Ettinger, Faeze Brahman, Sachin Kumar, Niloofar Mireshghallah, Ximing Lu, Maarten Sap, Yejin Choi, and Nouha Dziri. 2024. WildTeaming at Scale: From In-the-Wild Jailbreaks to (Adversarially) Safer Language Models. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. doi:10.52202/079017-1493

[6] Jayash Koshal and Shanu Sushmita. 2026. Cart Before the Harm: Multi-Turn Safety Detection in E-Commerce Conversational Agents. (May 2026). Unpublished project proposal manuscript, dated May 29, 2026.

[7] P. Li, J. Zhang, T. Zhang, H. Qiu, K. Zhang, W. Zhang, N. Yu, and W. Zhou. 2026. State-Dependent Safety Failures in Multi-Turn Language Model Interaction. arXiv:2603.15684 arXiv preprint.

[8] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. 2024. HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research,*

Vol. 235). PMLR, 35181–35224. https://proceedings.mlr.press/v235/mazeika24a.html

[9] Itay Nakash, George Kour, Koren Lazar, Matan Vetzler, Guy Uziel, and Ateret Anaby Tavor. 2025. Effective Red-Teaming of Policy-Adherent Agents. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Suzhou, China, 2250–2268. doi:10.18653/v1/2025.emnlp-main.114

[10] Qibing Ren, Hao Li, Dongrui Liu, Zhanxu Xie, Xiaoya Lu, Yu Qiao, Lei Sha, Junchi Yan, Lizhuang Ma, and Jing Shao. 2025. LLMs know their vulnerabilities: Uncover Safety Gaps through Natural Distribution Shifts. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 24763–24785. doi:10.18653/v1/2025.acl-long.1207

[11] Paul Röttger, Hannah Kirk, Bertie Vidgen, Giuseppe Attanasio, Federico Bianchi, and Dirk Hovy. 2024. XSTest: A Test Suite for Identifying Exaggerated Safety Behaviours in Large Language Models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Association for Computational Linguistics, 5377–5400. doi:10.18653/v1/2024.naacl-long.301

[12] Mark Russinovich, Ahmed Salem, and Ronen Eldan. 2025. Great, Now Write an Article About That: The Crescendo Multi-Turn LLM Jailbreak Attack. In *34th USENIX Security Symposium (USENIX Security 25)*. USENIX Association, 2421–2440. https://www.usenix.org/conference/usenixsecurity25/presentation/russinovich

[13] Anushka Sheoran and Yiduo Hao. 2026. MultiTurnPSB: Evaluating Multi-Turn Jailbreak Attacks and Classifier-Based Defenses for Medical AI Safety. arXiv:2606.02630 arXiv preprint.

[14] Tianyu Shi, Yang Mo, Yiou Liu, Zhuonan Hao, Yin Wang, Wenzhuo Hu, Nan Yu, Meng Zhou, and Jiangbo Yu. 2026. Organizational Control Layer: Governance Infrastructure at the Execution Boundary of LLM Agent Systems. arXiv:2606.04306 arXiv preprint.

[15] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2025. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In *International Conference on Learning Representations (ICLR 2025)*. https://proceedings.iclr.cc/paper_files/paper/2025/hash/1b126cc38b8638e07bef37e7b2bb72bf-Abstract-Conference.html

[16] Zhouwei Zhai, Mengxiang Chen, and Anmeng Zhang. 2026. SIA: A Synthesize-Inject-Align Framework for Knowledge-Grounded and Secure E-commerce Search LLMs with Industrial Deployment. arXiv:2603.16137 arXiv preprint.

[17] Zhou et al. 2024. Speak Out of Turn: Safety Vulnerability of Large Language Models in Multi-turn Dialogue. arXiv:2402.17262 arXiv preprint.